

Contents

Accuracy Benchmarking Plugin — Implementation Plan 1

Executive Summary 1

Table of Contents 2

1. Architecture Overview 2

2. Mode-Aware Pipeline Design 4

3. Component Diagram 7

4. Data Flow 8

5. New Plugin Categories 9

6. PR Breakdown 10

7. File Structure 12

8. Configuration Schema 13

9. Dataset Acquisition Strategy 14

10. CLI Interface 16

11. Example Usage 17

12. Testing Strategy 18

13. Risks and Mitigations 19

Appendices 20

Accuracy Benchmarking Plugin — Implementation Plan

Field	Value
Author	Claude Code
Date	February 2026
Status	Proposal
Timeline	~11 days (12 PRs)

Executive Summary

This document outlines the implementation plan for adding accuracy benchmarking capabilities to AIPerf, achieving **full feature parity** with LLM-CAPE’s accuracy evaluation system.

Key Benefits

- Yes Simultaneous performance AND accuracy metrics in a single benchmark run
- Yes Pluggable graders for different benchmark types
- Yes Native integration with AIPerf’s distributed architecture
- Yes **Full LLM-CAPE parity**: 9 benchmarks, 7 CLI options
- Yes **Direct HuggingFace integration**: No external framework dependencies

Scope

Category	Details
Benchmarks	MMLU, AIME, HellaSwag, BigBench, AIME24, AIME25, MATH-500, GPQA Diamond, LiveCodeBench
Graders	exact_match, math (SymPy), multiple_choice, code_execution (sandboxed)
Timeline	~11 days across 12 PRs

[!IMPORTANT] **Design Principle:** Accuracy mode is a SUPERSET of performance mode. - Accuracy benchmarking always provides performance metrics - Performance benchmarking cannot provide accuracy metrics (no ground truth) - The pipeline is mode-aware and conditionally activates accuracy components

Table of Contents

1. [Architecture Overview](#)
2. [Mode-Aware Pipeline Design](#)
3. [Component Diagram](#)
4. [Data Flow](#)
5. [New Plugin Categories](#)
6. [PR Breakdown](#)
7. [File Structure](#)
8. [Configuration Schema](#)
9. [Dataset Acquisition Strategy](#)
10. [CLI Interface](#)
11. [Example Usage](#)
12. [Testing Strategy](#)
13. [Risks and Mitigations](#)

1. Architecture Overview

Current AIPerf Architecture (9 Services)

Services: - **SystemController** — Orchestration, lifecycle management - **DatasetManager** — Prompt/token generation - **TimingManager** — Request scheduling, credit issuance - **WorkerManager** — Worker lifecycle, health monitoring - **Worker (N)** — LLM API calls, conversation state - **RecordProcessor (N)** — Metric computation - **RecordsManager** — Record aggregation - **GPUTelemetryManager** — GPU telemetry from DCGM - **ServerMetricsManager** — Server metrics from Prometheus

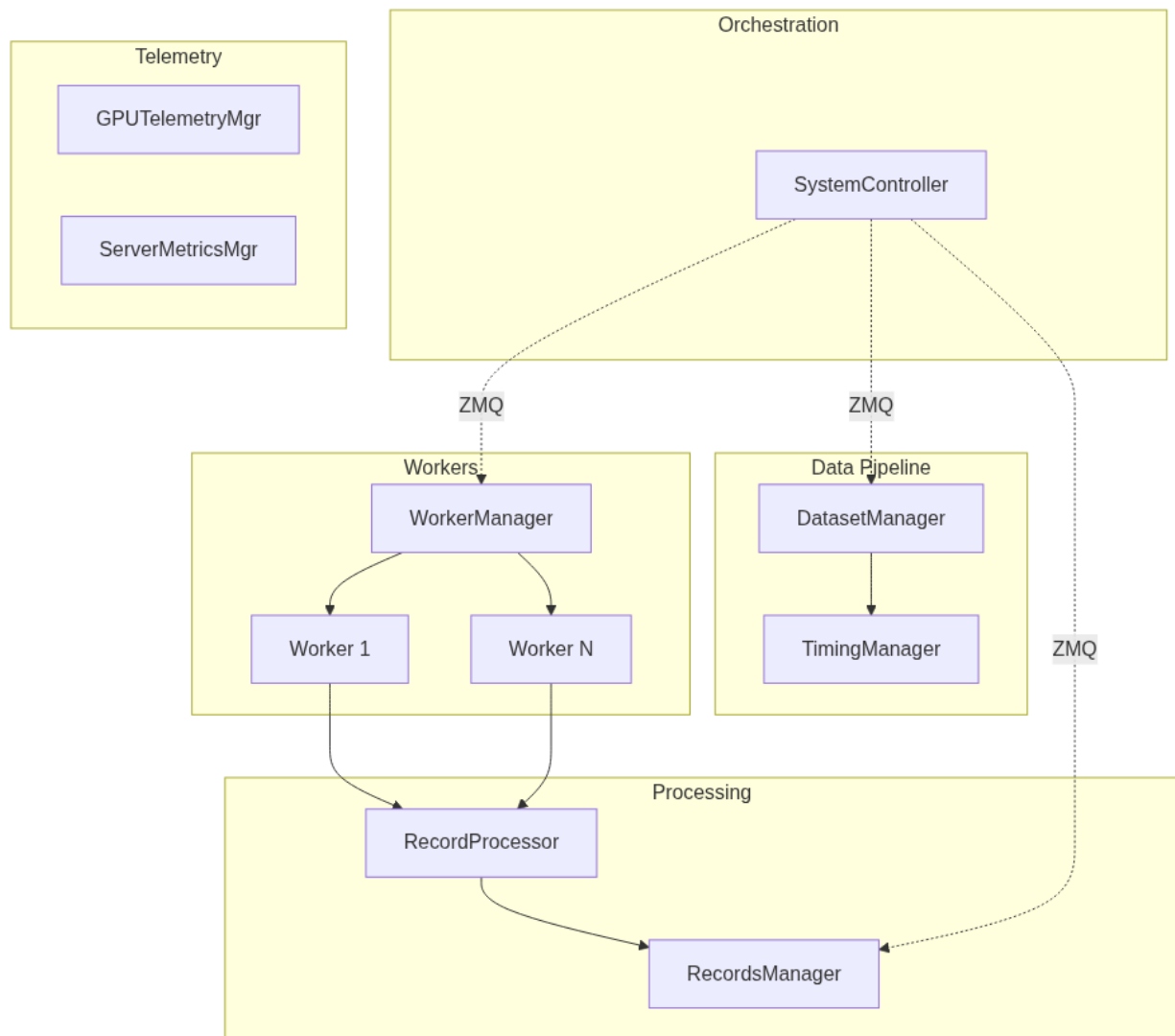


Figure 1: Diagram 1

New Architecture with Accuracy Plugin

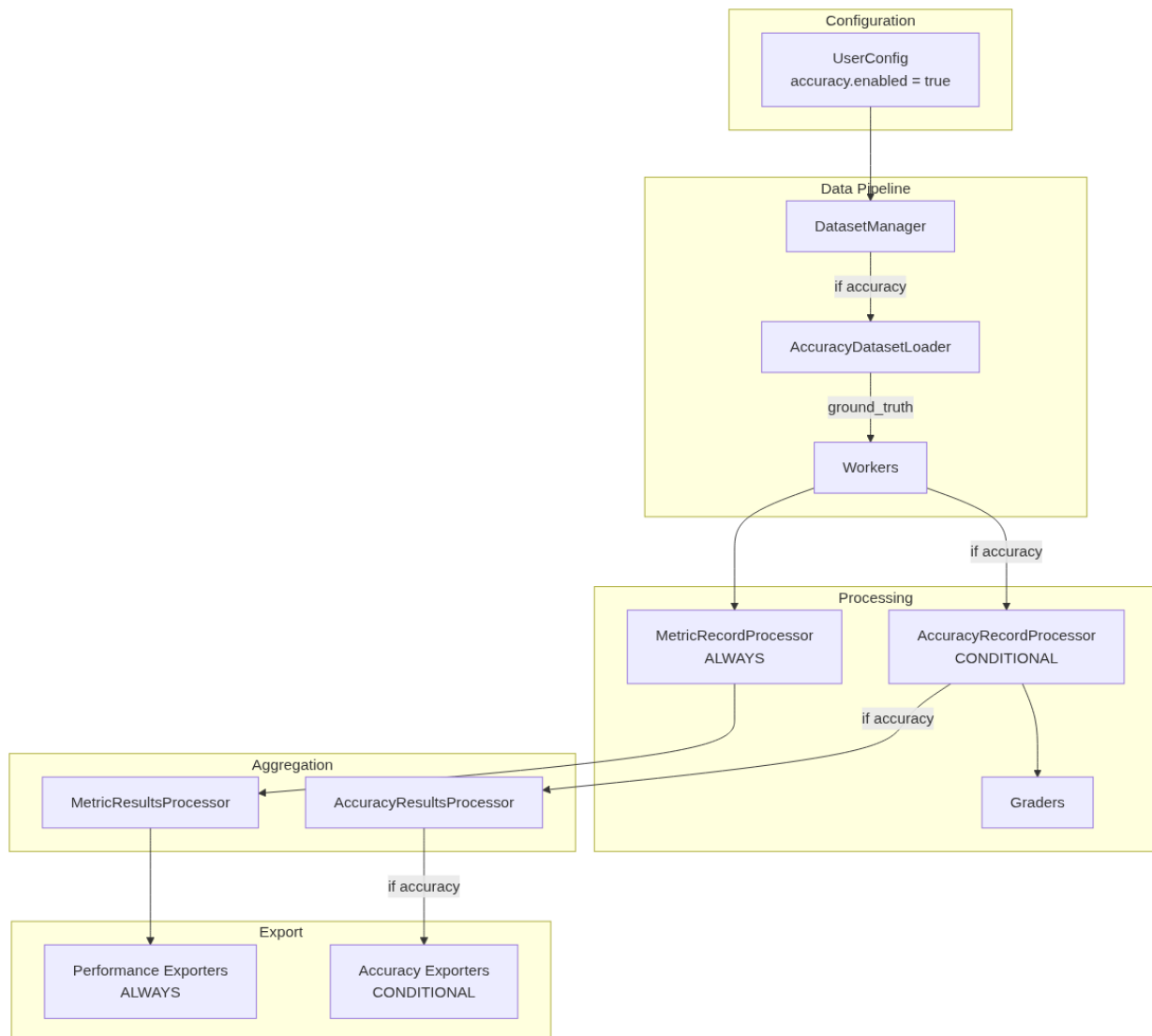


Figure 2: Diagram 2

2. Mode-Aware Pipeline Design

Key Design Principle

Mode	Performance Metrics	Accuracy Metrics
Performance Mode (accuracy.enabled = false)	Yes Yes	No No (no ground truth)
Accuracy Mode (accuracy.enabled = true)	Yes Yes	Yes Yes

Pipeline Builder Logic

```
def build_record_processors(user_config: UserConfig) -> list[RecordProcessorProtocol]:
    """Build list of active record processors based on config."""
    processors = []

    # ALWAYS include performance metrics processor
    processors.append(
        plugins.get_class(PluginType.RECORD_PROCESSOR, "metric_record")(user_config)
    )

    # CONDITIONALLY include accuracy processor
    if user_config.accuracy and user_config.accuracy.enabled:
        grader_type = user_config.accuracy.grader or _get_default_grader(
            user_config.accuracy.benchmark
        )
        processors.append(
            plugins.get_class(PluginType.RECORD_PROCESSOR, "accuracy_record")(
                user_config, grader_type=grader_type
            )
        )

    return processors
```

Results Model

```
class BenchmarkResults(AIPerfBaseModel):
    """Aggregated benchmark results."""

    # Performance metrics (ALWAYS present)
    performance: PerformanceMetrics = Field(
        description="Performance metrics (throughput, latency, TTFT, ITL, etc.)"
    )

    # Accuracy metrics (ONLY present in accuracy mode)
```

```

accuracy: AccuracyMetrics | None = Field(
    default=None,
    description="Accuracy metrics - only populated when accuracy benchmark enabled"
)

@property
def has_accuracy_metrics(self) -> bool:
    """Check if accuracy metrics are available."""
    return self.accuracy is not None

```

Console Output Examples

Performance Mode Output

```
$ aiperf profile --url http://localhost:8000 --endpoint-type chat --request-count 100
```

AIPerf v0.6.0 - LLM Inference Benchmark

Running benchmark (performance mode)...

```
[████████████████████████████████████████] 100% [100/100] 0:24
```

Performance Metrics

Request Throughput:	12.4 req/s
Output Token Throughput:	1,860 tokens/s
TTFT (p50/p90/p99):	45ms / 89ms / 142ms
ITL (p50/p90/p99):	12ms / 24ms / 38ms
Request Latency (p50):	1,247ms

Results saved to: ./results/profile_export_aiperf.json

Accuracy Mode Output

```
$ aiperf profile --url http://localhost:8000 --endpoint-type chat \
  --accuracy-benchmark mmlu --accuracy-n-shots 5 --request-count 500
```

AIPerf v0.6.0 - LLM Inference Benchmark

Loading MMLU benchmark (5-shot)...

Loaded 500 problems from cais/mmlu

Running benchmark (accuracy mode)...

```
[████████████████████████████████████████] 100% [500/500] 2:34
```

Performance Metrics

Request Throughput: 3.24 req/s
Output Token Throughput: 486 tokens/s
TTFT (p50/p90/p99): 124ms / 289ms / 412ms
ITL (p50/p90/p99): 18ms / 32ms / 48ms
Request Latency (p50): 2,847ms

Accuracy Results

Benchmark: MMLU (5-shot)
Overall Accuracy: 78.4% (392/500)
95% Confidence: [74.6%, 81.9%]

Per-Task Breakdown:

high_school_mathematics	72.0%	(180/250)
college_physics	84.8%	(212/250)

Results saved to: ./results/profile_export_aiperf.json
Accuracy details: ./results/profile_export_accuracy.csv

3. Component Diagram

Accuracy Graders (4 types)

Grader	Description	Use Case
exact_match	Case-insensitive string matching with whitespace normalization	Simple factual answers
math	SymPy-based symbolic comparison, \boxed{} extraction, numerical tolerance	AIME, MATH-500
multiple_choice	A/B/C/D/E extraction from various response formats	MMLU, HellaSwag, GPQA
code_execution	Sandboxed Python execution with output comparison	LiveCodeBench

Accuracy Benchmarks (9 total)

DeepEval Framework (4):

Benchmark	Tasks	Default Grader	Features
mmlu	57 subjects	multiple_choice	n-shot prompting
aime	Math competition	math	CoT support, max 8-shot

Benchmark	Tasks	Default Grader	Features
hellaswag	Commonsense	multiple_choice	Sentence completion
bigbench	Reasoning	multiple_choice	CoT support

LightEval Framework (5):

Benchmark	Source	Default Grader	Features
aime24	HuggingFaceH4/aime_2024	math	AIME 2024 problems
aime25	yentinglin/aime_2025	math	AIME 2025 problems
math_500	HuggingFaceH4/MATH-500	math	500 math problems
gpqa_diamond	ldavidrein/gpqa	multiple_choice	Graduate-level STEM
lcb_codegen	LiveCodeBench	code_execution	Code generation

4. Data Flow

Request Lifecycle with Accuracy

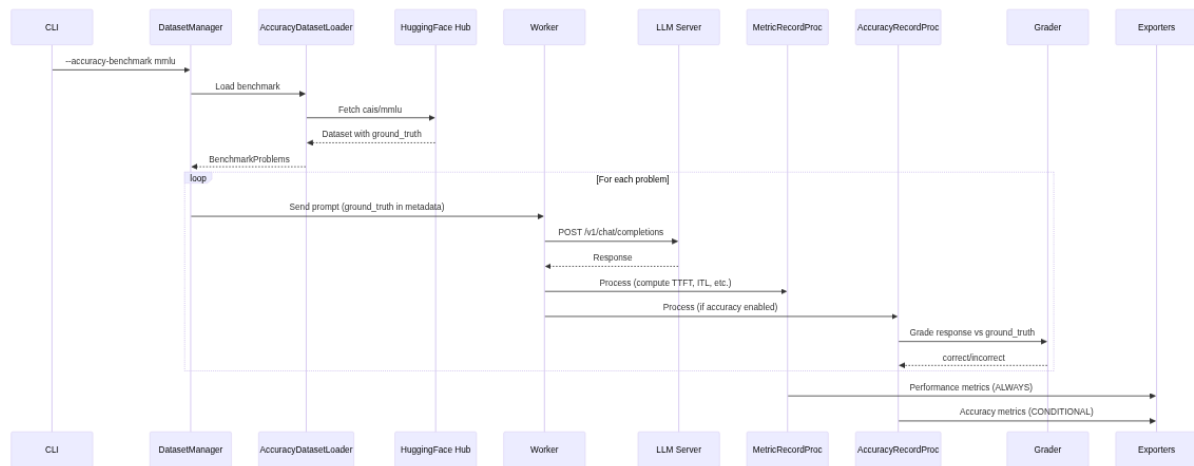


Figure 3: Diagram 3

Dataset Entry Format

```

{
  "prompt": "Question: What is 2+2?\nA) 3\nB) 4\nC) 5\nD) 6\nAnswer:",
  "request_id": "mmlu_math_001",
  "metadata": {
    "ground_truth": "B", # NOT sent to model
    "task": "high_school_mathematics",
    "benchmark": "mmlu"
  }
}

```

```
}  
}
```

5. New Plugin Categories

plugins.yaml Additions

```
# Accuracy Graders  
accuracy_grader:  
  exact_match:  
    class: aiperf.accuracy.graders.exact_match_grader:ExactMatchGrader  
    description: Case-insensitive exact string matching  
    metadata:  
      case_sensitive: false  
      strip_whitespace: true  
  
  math:  
    class: aiperf.accuracy.graders.math_grader:MathGrader  
    description: SymPy-based math grading with \boxed{} extraction  
    metadata:  
      numerical_tolerance: 1e-4  
      timeout_seconds: 3  
  
  multiple_choice:  
    class: aiperf.accuracy.graders.multiple_choice_grader:MultipleChoiceGrader  
    description: Extract A/B/C/D/E from response text  
    metadata:  
      valid_choices: ["A", "B", "C", "D", "E"]  
  
  code_execution:  
    class: aiperf.accuracy.graders.code_execution_grader:CodeExecutionGrader  
    description: Sandboxed code execution for LiveCodeBench  
    metadata:  
      timeout_seconds: 30  
      sandbox: true  
  
# Accuracy Benchmarks  
accuracy_benchmark:  
  mmlu:  
    class: aiperf.accuracy.benchmarks.mmlu_benchmark:MMLUBenchmark  
    metadata:  
      grader: multiple_choice  
      n_shots_default: 5  
      source: cais/mmlu
```

```

aime:
  class: aiperf.accuracy.benchmarks.aime_benchmark:AIMEBenchmark
  metadata:
    grader: math
    n_shots_default: 8
    supports_cot: true

# ... (other benchmarks follow same pattern)

```

6. PR Breakdown

Dependency Graph

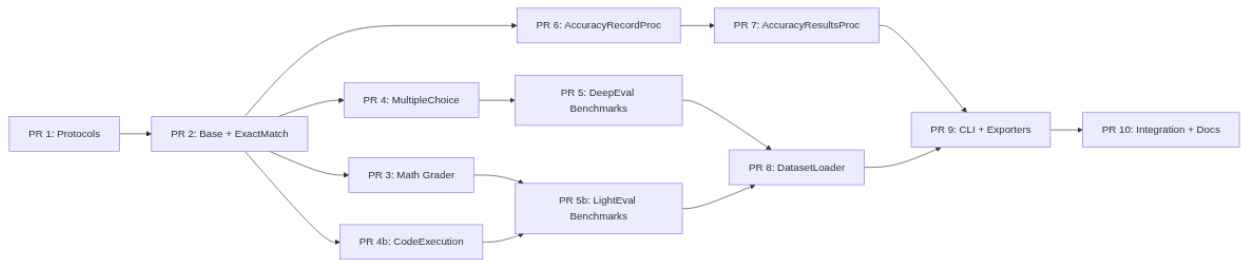


Figure 4: Diagram 4

Summary Table

PR	Title	Size	Time	Risk
1	Protocols & Categories	Small	0.5d	Low
2	Base + Exact Match Grader	Small	0.5d	Low
3	Math Grader	Medium	1d	Medium
4	Multiple Choice Grader	Small	0.5d	Low
4b	Code Execution Grader	Medium	1d	High
5	DeepEval Benchmarks (4)	Medium	1.5d	Medium
5b	LightEval Benchmarks (5)	Medium	1.5d	Medium
6	Accuracy Record Processor	Medium	1d	Low
7	Accuracy Results Processor	Small	0.5d	Low
8	Accuracy Dataset Loader	Medium	1d	Low
9	CLI + Exporter + Pipeline	Medium	1.5d	Medium
10	Integration + Docs	Medium	1d	Low
Total			~11 days	

[!NOTE] PRs 4b and 5b were added for full LLM-CAPE parity (+2.5 days for 6 additional benchmarks and code execution grader).

Detailed PR Specifications

PR 1: Protocols & Categories

Files to Create: - src/aiperf/accuracy/__init__.py - src/aiperf/accuracy/protocols.py

Acceptance Criteria: - [] AccuracyGraderProtocol with grade() and extract_answer() methods - [] AccuracyBenchmarkProtocol with load_problems() method - [] GradingResult and BenchmarkProblem Pydantic models - [] BenchmarkResults model with optional accuracy field - [] Categories registered in categories.yaml - [] make validate-plugin-schemas passes

PR 2: Base + Exact Match Grader

Files to Create: - src/aiperf/accuracy/graders/__init__.py - src/aiperf/accuracy/graders/exact_match_grader.py

Acceptance Criteria: - [] ExactMatchGrader implements AccuracyGraderProtocol - [] Configurable case sensitivity - [] Whitespace normalization - [] 90%+ test coverage

PR 3: Math Grader

Files to Create: - src/aiperf/accuracy/graders/math_grader.py

Acceptance Criteria: - [] Extracts answers from \boxed{} notation - [] Numerical comparison with configurable tolerance - [] SymPy symbolic comparison - [] Process isolation with 3-second timeout - [] Graceful fallback when SymPy fails

PR 4: Multiple Choice Grader

Files to Create: - src/aiperf/accuracy/graders/multiple_choice_grader.py

Acceptance Criteria: - [] Extracts A/B/C/D/E from response text - [] Handles patterns: "The answer is B", "(B)", "B.", "B)" - [] Case-insensitive matching

PR 4b: Code Execution Grader

Files to Create: - src/aiperf/accuracy/graders/code_execution_grader.py

Acceptance Criteria: - [] Sandboxed Python code execution - [] Configurable timeout (default 30s) - [] Output comparison with expected results - [] Security isolation (no filesystem/network access)

PR 5: DeepEval Benchmarks

Files to Create: - src/aiperf/accuracy/benchmarks/mmlu_benchmark.py - src/aiperf/accuracy/benchmarks/aime_benchmark.py - src/aiperf/accuracy/benchmarks/hellaswag_benchmark.py - src/aiperf/accuracy/benchmarks/bigbench_benchmark.py

Acceptance Criteria: - [] All 4 benchmarks load from HuggingFace - [] n-shot prompt formatting - [] System prompt pass-through - [] Mock HuggingFace in tests

PR 5b: LightEval Benchmarks

Files to Create: - src/aiperf/accuracy/benchmarks/aime24_benchmark.py - src/aiperf/accuracy/benchmarks/aime25_benchmark.py - src/aiperf/accuracy/benchmarks/livecode_benchmark.py - src/aiperf/accuracy/benchmarks/gpqa_benchmark.py - src/aiperf/accuracy/benchmarks/math500_benchmark.py

Acceptance Criteria: - [] All 5 benchmarks load from HuggingFace - [] LaTeX expression extraction for math grading - [] Code execution integration for LiveCodeBench

PR 6-10: Processing, Export, Integration

See [Testing Strategy](#) for acceptance criteria.

7. File Structure

New Files

```
src/aiperf/
├── accuracy/                                # NEW MODULE
│   ├── __init__.py
│   ├── protocols.py                        # Protocols + data models
│   └── graders/
│       ├── __init__.py
│       ├── base_grader.py
│       ├── exact_match_grader.py
│       ├── math_grader.py
│       ├── multiple_choice_grader.py
│       └── code_execution_grader.py
│   └── benchmarks/
│       ├── __init__.py
│       ├── base_benchmark.py
│       ├── mmlu_benchmark.py              # DeepEval
│       ├── aime_benchmark.py              # DeepEval
│       ├── hellaswag_benchmark.py         # DeepEval
│       ├── bigbench_benchmark.py          # DeepEval
│       ├── aime24_benchmark.py            # LightEval
│       ├── aime25_benchmark.py            # LightEval
│       ├── math500_benchmark.py           # LightEval
│       ├── gpqa_benchmark.py              # LightEval
│       └── livecode_benchmark.py          # LightEval
│   └── processors/
│       ├── __init__.py
│       ├── accuracy_record_processor.py
│       └── accuracy_results_processor.py
├── dataset/loaders/
│   └── accuracy_dataset_loader.py
└── exporters/
```

```

├── accuracy_console_exporter.py
├── accuracy_csv_exporter.py
├── cli_commands/
├── datasets.py                    # Dataset management CLI

tests/unit/accuracy/              # Unit tests
tests/integration/                # Integration tests
docs/accuracy_benchmarking.md     # User documentation
examples/accuracy/                # Example configs

```

8. Configuration Schema

YAML Configuration

```

# Standard endpoint config
endpoint:
  type: chat
  url: http://localhost:8000
  model: llama-3.1-70b-instruct

# Accuracy benchmarking config
accuracy:
  enabled: true                # MODE SWITCH
  benchmark: mmlu              # Benchmark name
  tasks:                       # Optional: filter to specific tasks
    - high_school_mathematics
    - college_physics
  n_shots: 5                   # Few-shot examples (0-8)
  enable_cot: false            # Chain-of-thought prompting
  grader: multiple_choice      # Override default grader
  system_prompt: "Answer with a single letter (A, B, C, or D)."
  verbose: false               # Log individual predictions

# Standard request config
request:
  count: 500
  concurrency: 10

output:
  artifact_dir: ./results

```

Pydantic Model

```

class AccuracyConfig(BaseConfig):
    """Configuration for accuracy benchmarking."""

```

```

enabled: bool = Field(
    default=False,
    description="Enable accuracy benchmarking mode"
)
benchmark: str | None = Field(
    default=None,
    description="Benchmark name (mmlu, aime, hellaswag, bigbench, "
                "aime24, aime25, math_500, gpqa_diamond, lcb_codegeneration)"
)
tasks: list[str] | None = Field(
    default=None,
    description="Specific tasks to include (None = all)"
)
n_shots: int = Field(
    default=0, ge=0, le=8,
    description="Number of few-shot examples (0-8)"
)
enable_cot: bool = Field(
    default=False,
    description="Enable chain-of-thought prompting"
)
grader: str | None = Field(
    default=None,
    description="Override default grader"
)
system_prompt: str | None = Field(
    default=None,
    description="System prompt for response format specification"
)
verbose: bool = Field(
    default=False,
    description="Log individual predictions and grading results"
)

```

9. Dataset Acquisition Strategy

Design Decision

AlPerf uses the HuggingFace datasets library directly — no external framework dependencies (DeepEval/LightEval).

Benefits: - Yes Minimal dependencies (only datasets library) - Yes Full control over prompt formatting - Yes Same data sources as LLM-CAPE - Yes Native async integration

HuggingFace Dataset Sources

Benchmark	Repository	Split	Ground Truth
MMLU	cais/mmlu	test	answer (0-3)
HellaSwag	Rowan/hellaswag	validation	label
BigBench	maveriq/bigbenchhard	train	target
AIME	Maxwell-Jia/AIME_2024	train	Answer
AIME 2024	HuggingFaceH4/aime_2024	train	answer
AIME 2025	yentinglin/aime_2025	train	answer
MATH-500	HuggingFaceH4/MATH-500	test	answer
GPQA Diamond	Idavidrein/gpqa	train	answer
LiveCodeBench	livecodebench/code_generation_lite	test	expected_output

Dataset Loader Implementation

```

from datasets import load_dataset

class MMLUBenchmark(AccuracyBenchmarkProtocol):
    """MMLU benchmark loader using HuggingFace datasets directly."""

    HF_REPO = "cais/mmlu"

    async def load_problems(
        self,
        tasks: list[str] | None = None,
        n_shots: int = 5,
        limit: int | None = None,
        system_prompt: str | None = None,
    ) -> list[BenchmarkProblem]:
        # Load from HuggingFace (automatically cached)
        dataset = await asyncio.to_thread(
            load_dataset, self.HF_REPO, "all", trust_remote_code=True
        )

        # Build few-shot examples from dev split
        few_shot_examples = self._build_few_shot_examples(dataset["dev"], n_shots)

        # Convert test split to BenchmarkProblems
        problems = []
        for item in dataset["test"]:
            if tasks and item["subject"] not in tasks:
                continue

            problem = BenchmarkProblem(
                prompt=self._format_prompt(item, few_shot_examples, system_prompt),
                ground_truth=item["answer"],
                task=item["subject"],
                metadata={"benchmark": "mmlu", "n_shots": n_shots},
            )

```

```

    )
    problems.append(problem)

    if limit and len(problems) >= limit:
        break

    return problems

```

Caching & Offline Support

Cache location: ~/.cache/huggingface/datasets/

Environment variables:

```

export HF_DATASETS_CACHE="/path/to/cache" # Custom cache location
export HF_DATASETS_OFFLINE=1             # Offline mode

```

CLI commands:

```

# Pre-download datasets
aiperf datasets download --accuracy-benchmark mmlu

# List cached datasets
aiperf datasets list

# Clear cache
aiperf datasets clear --benchmark mmlu

```

Dataset Sizes

Benchmark	Size	Download Time (50 Mbps)
MMLU	~300 MB	~1 minute
HellaSwag	~50 MB	~10 seconds
BigBench	~100 MB	~20 seconds
AIME variants	~5 MB	~2 seconds
MATH-500	~20 MB	~5 seconds
GPQA Diamond	~10 MB	~3 seconds
LiveCodeBench	~50 MB	~10 seconds

10. CLI Interface

New Flags for aiperf profile

Accuracy Benchmarking Options:

--accuracy-benchmark TEXT Run accuracy benchmark. Available:

	DeepEval: mmlu, aime, hellaswag, bigbench
	LightEval: aime24, aime25, math_500,
	gpqa_diamond, lcb_codegeneration
--accuracy-tasks TEXT	Comma-separated tasks within benchmark
--accuracy-n-shots INTEGER	Few-shot examples [default: 0, max: 8]
--accuracy-enable-cot	Enable chain-of-thought prompting
--accuracy-grader TEXT	Override grader (exact_match, math, multiple_choice, code_execution)
--accuracy-system-prompt TEXT	System prompt for response format
--accuracy-verbose	Log individual predictions

Usage Examples

```
# Performance mode (default)
aipperf profile --url http://localhost:8000 --endpoint-type chat

# MMLU with 5-shot
aipperf profile --url http://localhost:8000 --endpoint-type chat \
  --accuracy-benchmark mmlu \
  --accuracy-n-shots 5 \
  --accuracy-system-prompt "Answer with a single letter (A, B, C, or D)."
```

```
# AIME with chain-of-thought
aipperf profile --url http://localhost:8000 --endpoint-type chat \
  --accuracy-benchmark aime \
  --accuracy-enable-cot \
  --accuracy-n-shots 8
```

```
# GPQA Diamond (graduate-level STEM)
aipperf profile --url http://localhost:8000 --endpoint-type chat \
  --accuracy-benchmark gpqa_diamond \
  --accuracy-verbose
```

```
# LiveCodeBench (code generation)
aipperf profile --url http://localhost:8000 --endpoint-type chat \
  --accuracy-benchmark lcb_codegeneration
```

11. Example Usage

MMLU Configuration File

```
# examples/accuracy/mmlu_config.yaml
endpoint:
  type: chat
  url: http://localhost:8000
  model: llama-3.1-70b-instruct
```

```

accuracy:
  enabled: true
  benchmark: mmlu
  tasks:
    - high_school_mathematics
    - college_physics
    - computer_science
  n_shots: 5
  system_prompt: "Answer with a single letter (A, B, C, or D)."
```

```

request:
  count: 500
  concurrency: 10
```

AIME Configuration File

```

# examples/accuracy/aime_config.yaml
endpoint:
  type: chat
  url: http://localhost:8000
  model: deepseek-r1

accuracy:
  enabled: true
  benchmark: aime
  n_shots: 8
  enable_cot: true
  verbose: true

request:
  count: 30
  concurrency: 5
```

12. Testing Strategy

Unit Tests

Component	Coverage Target
Protocols	95%
Exact Match Grader	95%
Math Grader	90%
Multiple Choice Grader	95%
Code Execution Grader	90%
Benchmarks	90%
Record Processor	90%

Component	Coverage Target
Results Processor	90%

Integration Tests

```
@pytest.mark.integration
async def test_mode_switching_no_state_leak():
    """Verify mode switching doesn't leak accuracy state."""
    # Run performance mode
    perf_results = await run_benchmark(accuracy_enabled=False)
    assert perf_results.accuracy is None

    # Run accuracy mode
    acc_results = await run_benchmark(accuracy_enabled=True, benchmark="mmlu")
    assert acc_results.accuracy is not None

    # Run performance mode again
    perf_results_2 = await run_benchmark(accuracy_enabled=False)
    assert perf_results_2.accuracy is None # No leakage
```

13. Risks and Mitigations

Risk	Impact	Likelihood	Mitigation
SymPy hangs on pathological input	High	Medium	Process isolation + 3s timeout
Code execution security	High	Medium	Sandboxed execution, no filesystem/network
HuggingFace Hub unavailable	Medium	Low	Local caching, offline mode, pre-download CLI
Dataset download on first run	Low	High	Progress indicator, pre-download command
Grader extraction failures	Medium	Medium	Fallback patterns, log warnings
Breaking existing pipeline	High	Low	Feature flag, mode-aware activation

Safety Implementations

SymPy Safety

```
from concurrent.futures import ProcessPoolExecutor, TimeoutError

def _safe_sympy_compare(pred: str, gt: str, timeout: int = 3) -> bool:
    """Run SymPy comparison in isolated subprocess with timeout."""
    with ProcessPoolExecutor(max_workers=1) as executor:
        future = executor.submit(_sympy_equal, pred, gt)
        try:
            return future.result(timeout=timeout)
        except TimeoutError:
            return False
```

Code Execution Safety

```
import subprocess
import tempfile

def _safe_code_execute(code: str, timeout: int = 30) -> tuple[str, bool]:
    """Execute code in isolated subprocess."""
    with tempfile.NamedTemporaryFile(mode='w', suffix='.py', delete=False) as f:
        f.write(code)
        temp_path = f.name

    try:
        result = subprocess.run(
            ['python', '-u', temp_path],
            capture_output=True,
            text=True,
            timeout=timeout,
            env={},          # Clean environment
            cwd='/tmp',      # Isolated directory
        )
        return result.stdout, result.returncode == 0
    except subprocess.TimeoutExpired:
        return "", False
    finally:
        os.unlink(temp_path)
```

Appendices

Appendix A: LLM-CAPE Comparison

Aspect	LLM-CAPE	AIPerf Plugin
Framework	Wraps deepeval/lighteval	Native implementation
Architecture	Single process	Distributed (ZMQ)
Grading	Synchronous	Async-compatible
Configuration	JSON	YAML + CLI
Combined metrics	No (separate runs)	Yes (same run)
Benchmarks	9	9 (full parity)
CLI Options	7	7 (full parity)

Appendix B: Dependencies

Add to `pyproject.toml`:

```
[project.optional-dependencies]
accuracy = [
    "datasets>=2.14.0",
    "huggingface_hub>=0.23.0",
    "sympy>=1.12",
]
```

Install: `pip install aiperf[accuracy]`

Appendix C: Future Enhancements

1. Semantic similarity grader (sentence transformers)
2. Partial credit grading (0.0-1.0 scores)
3. Accuracy timeslices (track accuracy over time)
4. A/B model comparison mode
5. Accuracy-aware goodput (SLO-based metrics)
6. Custom user-defined benchmarks (JSONL format)
7. HumanEval benchmark
8. MT-Bench (multi-turn evaluation)

Build systems that scale. Write code that lasts.